Operational Test: The Conflict-Coexistence Test as Standalone Procedure Specification in the Coordination Knowledge Substrate Pattern

Wenxin Li (Independent Researcher)
ORCID: 0009-0004-8065-3235
May 7, 2026

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to formalize the **conflict-coexistence test** — the operational procedure that verifies a CKS deployment satisfies the substrate-accommodates-conflicting-state commitment within the broader conflict-as-first-class commitment — as a standalone, executable specification with defined procedure, defined outputs, and defined detected anti-patterns.

Abstract

The CKS pattern's conflict-as-first-class commitment decomposes into operational variants. One of them — *substrate accommodates conflicting state* — names the architectural promise that the substrate accepts contradictory writes about the same entity without forcing automatic reconciliation, registers the contradiction as substrate-resident state per the source paper's authoritative-content categories (§11.3), distinguishes the conflicting writes through provenance fields the path-retraceability commitment requires (§3.1), and defers resolution to humans authoring rules under their preserved governance authority (§2.1, §3.3). The commitment can be made true by an architecture and false by a deployment built on top of it; the deployment must be tested. This note formalizes the conflict-coexistence test as the standalone operational procedure that performs that test: the architectural commitment under test, the procedure as an ordered sequence of insertions and observations, the pass/fail outputs, the canonical anti-patterns it detects (forced reconciliation, vendor-managed conflict, contradiction collapse by automation, plus four narrower variants common in LLM-agent deployments), the integration points with deployment-verification workflow, and the explicit limits of what the test does and does not verify. The note closes the substrate-operational-properties cluster covering tool-agnosticism migration, linear-cost scaling, and conflict coexistence; the operational-tests phase then turns to composition tests.

1. Why the conflict-coexistence test needs to be formalized as standalone

The conflict-as-first-class commitment is the architectural move that distinguishes CKS most sharply from the dominant LLM-agent design pattern. Most contemporary systems treat contradictions as

exceptions to be resolved before further processing — by clarification turns, judge-model arbitration, last-write-wins defaults, vendor-managed consensus, or the silent merging that occurs when an LLM is asked to "summarize" two contradicting inputs. CKS commits to the opposite stance: contradictions are routine substrate state that the architecture must accommodate without forcing collapse, and resolution — when it occurs — runs under human-authored rules, not under the substrate's defaults or an LLM's judgment (§5, §11.3 of the source paper).

The architectural commitment is one thing; the deployment that implements it is another. A deployment can claim conflict-as-first-class while in fact implementing a substrate that quietly enforces last-write-wins, registers conflicts in a vendor-managed system outside substrate scope, or routes contradictions through an LLM that decides which one "wins." Each of these failure modes preserves the surface vocabulary of conflict handling while breaking the architectural commitment. The only way to distinguish a deployment that satisfies the substrate-accommodates-conflicting-state commitment from one that nominally claims to is to insert contradictions and observe what the substrate actually does.

The conflict-coexistence test formalizes that procedure, and is published as a standalone derivation note because the operational content of "verify conflict accommodation in a deployment" is independent of the commitment's specification: a deployment passes or fails on its observed behavior under contradictory writes, regardless of what its architectural documentation claims. The note also closes the substrate-operational-properties cluster of the operational-tests phase — the third entry in a triplet covering portability, cost behavior, and conflict accommodation, the operational properties most often misimplemented under load.

2. The architectural commitment under test

The conflict-coexistence test verifies one specific commitment: *substrate accommodates conflicting state*, as a decomposition of conflict-as-first-class. The commitment requires the following five properties to hold at all times during the substrate's existence.

(a) The substrate accepts contradictory writes. When two writes about the same entity carry contradictory content, both writes are accepted into substrate state. Neither is rejected by the substrate's defaults; neither silently overwrites the other.

(b) The contradiction is registered as substrate-resident authoritative content. Per the source paper's §11.3 enumeration of substrate authoritative-content categories, the contradiction relationship — "these two writes contradict on this dimension" — lives in the substrate. The conflict registry is substrate-resident, not vendor-resident, not LLM-mediated, not maintained as a transient signal in some adjacent system.

(c) Provenance distinguishes the conflicting writes. Per the path-retraceability commitment (§3.1), each write carries the substrate's provenance fields: writer identity, timestamp, rule-of-authorship reference where applicable, cell-execution identifier where applicable, rationale where applicable, and relationship to the substrate state it modifies. The two conflicting writes must be distinguishable from each other by provenance, not conflated into a single record.

(d) No automatic reconciliation occurs. The substrate does not collapse the contradiction by last-write-wins defaults, vendor-managed consensus algorithms, LLM-mediated decision, or any other automated process. Both contradictory writes remain in substrate state until a human acts.

(e) Human-authored rules are the resolution path. When resolution occurs, it occurs because a human has authored an orchestration rule specifying how the conflict is to be resolved, or because a human has directly modified substrate content under override authority. The resolution rule is itself substrate-resident authoritative content; the resolution decision is recorded as new substrate state, with the prior conflicting facts retained in provenance.

These five properties together compose the commitment. The conflict-coexistence test verifies all five.

3. The test procedure

The test procedure consists of six ordered steps. Each step performs one operation and observes one property.

Step 1 — Insert two contradictory facts. Through two separate cell executions, or two direct human modifications under the modify right, insert two facts about the same entity that contradict on at least one dimension. Both writes follow the deployment's normal write path; the test does not exercise a special test mode.

Step 2 — Verify substrate acceptance of both writes. Read substrate state. Confirm that neither write has been rejected, neither has silently overwritten the other, and both are present with their full content intact. A deployment that rejects either write or reduces both to a single record fails this step.

Step 3 — Verify substrate-resident conflict registration. Query the substrate's conflict registry — whatever mechanism the deployment uses to address the contradiction relationship — and confirm that the contradicting state pair is registered as substrate-resident authoritative content, queryable within the substrate's normal access path. A registration that lives only in a vendor monitoring system, an LLM context buffer, or a runtime middleware layer fails this step.

Step 4 — Verify provenance distinguishes the writes. For each of the two writes, read the provenance fields and confirm distinguishability on at least the writer-identity, timestamp, and rule-of-authorship dimensions. Where the writes were performed by cells, the cell-execution identifier must distinguish them; where by humans directly, the writer-identity field must do so. Provenance that conflates the two writes fails this step.

Step 5 — Verify no automatic reconciliation has occurred. Wait for whatever interval the deployment specifies as its reconciliation horizon, or — if none is named — a deployment-appropriate observation window. Re-read substrate state and confirm that both contradictory facts remain, that the conflict registration is still present, and that no automated process has produced a reconciled record. A deployment in which the substrate, the LLM, vendor consensus algorithms, or runtime middleware has resolved the conflict during this interval fails this step.

Step 6 — Verify human-authored resolution succeeds. A human, exercising the rule-authoring right, authors an orchestration rule that specifies how the conflict is to be resolved (or directly modifies substrate content under override authority). After the rule takes effect, confirm that the resolution decision is recorded as new substrate content, that the prior conflicting facts remain in provenance, and that the resolution rule is itself substrate-resident authoritative content. A deployment that loses the prior conflicting facts during resolution, or that places the resolution rule outside substrate scope, fails this step.

The test passes when all six steps pass. The test fails when any step fails, with the failed step naming the specific property the deployment does not satisfy.

4. What the test outputs

The test produces a binary pass/fail output, with a structured failure report when applicable.

Pass. Both contradictory writes are accepted (Step 2); the conflict is registered substrate-resident (Step 3); provenance distinguishes the writes (Step 4); no automatic reconciliation has occurred (Step 5); human-authored resolution succeeds with prior facts retained in provenance (Step 6).

Fail. One write is rejected, conflated, or silently overwritten (Step 2); the conflict registry lives outside substrate scope or does not exist (Step 3); the conflicting writes are not distinguishable by provenance (Step 4); automatic reconciliation has occurred — by substrate default, vendor algorithm, LLM mediation, or runtime middleware (Step 5); human-authored resolution produces a state in which prior facts are lost or the resolution rule is non-substrate-resident (Step 6).

The pass output certifies that the deployment satisfies the substrate-accommodates-conflicting-state commitment on the observed behaviors. It does not certify other conflict-as-first-class commitments not specifically tested here (see §7). The fail output names the property failed and the anti-pattern the failure most likely instantiates.

5. What anti-patterns the test specifically detects

The test detects three canonical anti-patterns named in the broader anti-pattern formalizations, plus four narrower variants particularly common in LLM-agent deployments.

Forced reconciliation (Step 2 fails): the substrate forces automatic reconciliation rather than accepting both contradictory writes. The forcing may be implemented as a uniqueness constraint, a transactional merge, an upsert default, or any other mechanism that reduces two contradictory writes to one record at write time.

Vendor-managed conflict (Step 3 fails): the conflict registry lives in vendor systems — a database vendor's internal conflict-detection facility, a SaaS application's merge-management layer, a synchronization tool's deduplication engine — rather than in substrate-resident authoritative content. The contradiction may be detected and even surfaced, but it is not addressable as substrate state under the human-governance authority architecture.

Contradiction collapse by automation (Step 5 fails): an LLM, an automated process, or a runtime middleware layer resolves the contradiction during the observation window without a human-authored rule explicitly authorizing the specific resolution. The collapse may be motivated by helpfulness, performance, or convention; the architectural commitment is violated regardless of motive.

Four narrower variants, each a specific instance of one of the above:

bullet **Last-write-wins-without-rule** (Step 5): the substrate's default allows later writes to silently overwrite earlier ones, with no explicit human-authored rule specifying that resolution logic. Last-write-wins as an explicit rule is admissible; as an unstated default it is not.

bullet **LLM-mediated conflict resolution** (Step 5): an LLM call determines which conflicting fact "wins." The LLM is permissible as a tool human operators may consult while authoring resolution rules; it is not permissible as the gate through which contradictions are silently resolved.

bullet **Vendor-managed reconciliation** (Step 5): a vendor-supplied consensus algorithm — eventual consistency, three-way merge, distributed transaction — resolves the contradiction without invoking human-authored substrate rules. The algorithms may serve other purposes (replication, availability under partition); they are not the resolution path the architectural commitment names.

bullet **Conflict-registry-in-external-system** (Step 3): the contradiction's registration lives outside substrate scope — a separate audit log, a monitoring dashboard, an LLM context buffer — even though both contradicting writes are present in substrate. The contradiction relationship is itself authoritative content per §11.3 and must live where authoritative content lives.

Each failure has the same architectural shape: contradictions, their provenance, or their resolution path is held somewhere the human-governance authority architecture cannot reach. The test detects each by exercising the path the failure would short-circuit.

6. How the test integrates with deployment verification

The test is intended to be run at five points in a deployment's lifecycle.

Initial deployment validation. Before activation, the test is run with synthetic contradictory writes to confirm the substrate accommodates conflicts as the commitment requires. A failed validation must be remediated before activation; the failed step names what to fix.

Post-conflict-incident verification. When a real contradiction occurs in deployed operation, the test is rerun against the actual incident — verifying that the live conflict was registered substrate-resident, that provenance distinguishes the writes, that no automatic reconciliation occurred, and that resolution (when it occurred) ran through a human-authored rule. Real incidents are the highest-fidelity test material.

Composition-partner verification. When two CKS substrates compose, the test is run across the composition boundary to verify that conflicts arising at the seam are accommodated by the composed system as a whole. Composition that introduces forced reconciliation at the seam fails the commitment

even when each substrate alone satisfies it.

Vendor-migration verification. After a host-environment change — between database vendors, substrate hosts, or underlying storage technologies — the test is rerun to verify the migration did not introduce auto-reconciliation as a side effect of the new host's defaults. Many platform-level facilities ship with last-write-wins or vendor-managed merge as defaults; a migration that preserves substrate content while losing conflict accommodation is a common silent regression.

LLM-vendor-update verification. When the LLM the deployment uses is updated, the test is rerun to verify that the new LLM does not perform automatic conflict resolution where the prior LLM did not. The architectural commitment is to the substrate's behavior, not the LLM's, but LLM behavioral change is among the most common ways the commitment silently regresses, because new model versions often "improve" by being more decisive in the face of contradictions.

These five points share a structure: each is a moment at which the deployment's conflict-handling guarantees are most likely to drift, and therefore a moment at which the standalone test produces high-value evidence.

7. Limits of the test

The test is narrow by design. A deployment that passes may still fail commitments the test was not built to detect.

It does not verify other conflict-as-first-class aspects in isolation — the integrating-frame variant, the registration-mechanism variant, the conflict-has-provenance variant, the resolution-through-rule-authoring variant. The test covers them only insofar as testing substrate-accommodates-conflicting-state depends on them; standalone verification of the others is the responsibility of separate operational-test notes.

It does not verify conflict-handling determinism. The determinism contract names a guarantee that same conflicts produce same registry entries with same provenance across runs; verifying that requires running the procedure multiple times under controlled conditions and observing identical outputs — the procedural shape of the cell-behavior-determinism test rather than this one.

It does not verify conflict appropriateness or resolution-rule correctness. A deployment that accommodates a contradiction between "the customer's address is in Boston" and "the customer's address is on Mars" passes as fully as one in which the contradiction is between two plausible contact preferences. The test verifies architectural accommodation, not whether the contradiction is meaningful, well-posed, or worth preserving. A human-authored resolution rule that produces a domain-incorrect outcome can still satisfy Step 6: domain correctness is the responsibility of the humans authoring the rules.

It does not verify substrate-content correctness or the broader human-governed commitment. Provenance fields may be populated with incorrect values without the test detecting it, provided the fields are present and distinguish the writes. Step 6 confirms human-authored resolution succeeds in the case tested; standalone tests for inspect, modify, override, and rule authoring are separate.

These limits are deliberate. The test is meant to be a precise instrument that detects substrate-accommodates-conflicting-state violations cleanly, not a general-purpose architectural audit. Composing it with the other operational tests in the phase is what produces full coverage.

8. Operational test summary

A CKS deployment passes the conflict-coexistence test if and only if all of the following hold, observed under the procedure in §3:

1. Two contradictory writes about the same entity are both accepted into substrate state without rejection, conflation, or silent overwrite.
2. The contradiction relationship is registered as substrate-resident authoritative content, queryable within the substrate's normal access path.
3. Provenance distinguishes the two conflicting writes on at least the writer-identity, timestamp, and rule-of-authorship dimensions.
4. No automatic reconciliation occurs during the deployment's observation window — neither by substrate default, vendor consensus, LLM mediation, nor runtime middleware.
5. Human-authored resolution succeeds, with prior conflicting facts retained in provenance and the resolution rule itself substrate-resident.

A deployment that fails any of (1)–(5) does not satisfy the substrate-accommodates-conflicting-state commitment, regardless of how robustly it satisfies surrounding commitments. A deployment that passes (1)–(5) satisfies the commitment, observed on the test material, with the limits stated in §7.

9. Conclusion

The conflict-coexistence test formalizes the procedure that distinguishes a deployment claiming to accommodate contradictions from one that actually does. The distinction matters at the architectural layer because most LLM-agent deployments treat contradictions as exceptions to be resolved before further processing — a stance that breaks the conflict-as-first-class commitment as soon as the contradiction has any property worth preserving. The test detects the canonical breakings and four narrower LLM-agent variants by exercising the operational path each breaking would short-circuit.

The note closes the substrate-operational-properties cluster of the operational-tests phase: the prior notes covered tool-agnosticism migration (portability across host environments) and linear-cost scaling (cost behavior under substrate growth); this note covers conflict accommodation under contradiction. The three together exercise the substrate's load-bearing operational properties under the three pressures that typically produce silent regression — host migration, scale, and contradiction. The phase then turns to composition tests, beginning with the composition-requirements-five test.

Subsequent work that implements, extends, or argues against the CKS conflict-as-first-class commitment should run the test in the form specified here, or specify precisely how its variant differs and why the architectural commitment is preserved under the variant. Subsequent work that claims substrate-accommodates-conflicting-state satisfaction without running an equivalent test is making a

claim it has not verified.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems.* April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *Operational Test: The Conflict-Coexistence Test as Standalone Procedure Specification in the Coordination Knowledge Substrate Pattern.* May 7, 2026. ORCID: 0009-0004-8065-3235.